

## Apresentação pontos do jogo

### Echoes of the Forest

Aluno: Carlos Gabriel Teixeira

- **Introdução**

. Primeiro aqui estão os sprites do nosso personagem e do nosso inimigo:



Figura 1: Sprite do jogador.

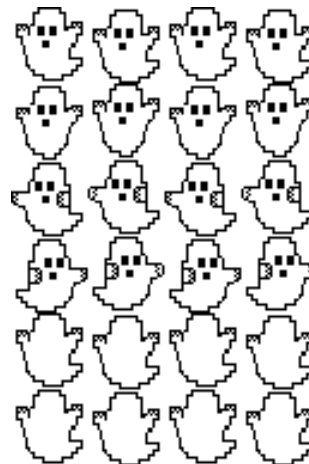


Figura 2: Sprite do fantasma.

- **Mecânicas**

. Agora, algumas das principais mecânicas do nosso projeto, aqui você encontra como cada um desses elementos foi feito, esses foram os maiores desafios do trabalho e o que consideramos como diferencial do nosso jogo.

#### 1. Jogador se movendo com sistema de lanterna:



Figura 3: Apresentação do sistema de lanterna.

- Aqui consta nosso personagem correndo (com animação) e sua lanterna ligada. Para elaborarmos o sistema de lanterna foi preciso de uma tela de escurecimento por todo o mapa enquanto calculamos um ângulo para ser iluminado quando o jogador ligar a lanterna. Abaixo colocamos a função que nos mostra o cálculo feito, usando a biblioteca math, o desenho da lanterna que precisa acompanhar o movimento da câmera e o desenho das camadas de luz. A lanterna conta com um som de liga/desliga.

```

def desenhar(self, screen, player_x, player_y, player_width, player_height, camera_x, camera_y):

    #Desenha a lanterna sobre a tela.

    # Cria a camada de escurecimento
    overlay = pygame.Surface(screen.get_size(), pygame.SRCALPHA)
    overlay.fill(self.escurecimento)

    # Calcula a posição da lanterna ajustada pela câmera
    keys = pygame.key.get_pressed()

    self.lanterna_pos_x = player_x + player_width // 2 - camera_x + 20
    self.lanterna_pos_y = player_y + player_height // 2 - camera_y + 30

    if self.ligada:
        # Direção da luz (baseada na posição do mouse)
        mouse_x, mouse_y = pygame.mouse.get_pos()
        dir_x, dir_y = mouse_x - self.lanterna_pos_x, mouse_y - self.lanterna_pos_y
        angle = math.atan2(dir_y, dir_x)

        # Cria os pontos do cone de luz
        points = [(self.lanterna_pos_x, self.lanterna_pos_y)]
        for i in range(-self.angulo // 2, self.angulo // 2 + 1, 5):
            end_x = self.lanterna_pos_x + int(self.raio * math.cos(angle + math.radians(i)))
            end_y = self.lanterna_pos_y + int(self.raio * math.sin(angle + math.radians(i)))
            points.append((end_x, end_y))

        # Desenha o cone de luz
        pygame.draw.polygon(overlay, self.cor_luz, points)

    # Aplica o overlay à tela
    screen.blit(overlay, (0, 0))

```

Figura 4: Função de desenhar lanterna

## 2. Condição de vitória = Chave:



Figura 5: Jogador encontra a chave

- Essa foi uma mecânica simples, mas que consideramos eficiente e um diferencial para o jogo pois é o objetivo do jogador durante a gameplay. A função para chave é simples, ela desenha o objeto e some no momento em que o personagem e o mesmo colidirem, nesse momento é possível ouvir um som de coleta.

### 3. Aparição do fantasma:

Encontre a chave da cabana e retorne



Figura 6: Exemplo com fantasma não aparecendo.

Encontre a chave da cabana e retorne



Figura 7: Exemplo com fantasma aparecendo.

- Essa é a mecânica mais complicada e a que mais gostamos de fazer, é a que apresenta o maior diferencial do nosso jogo. Nós precisamos de uma condição de desenhar o fantasma caso sua posição esteja dentro do raio da lanterna enquanto ela está acessa, para isso foi preciso calcular a distância do fantasma com o jogador, além do ângulo de visão do jogador que será usada para calcular a diferença angular, isso foi útil para o fantasma aparecer não somente quando estiver dentro da distância exigida, mas quando a lanterna estiver apontada para ele também.

```
# Configuração para o fantasma só aparecer quando a lanterna apontar para ele
mouse_x, mouse_y = pygame.mouse.get_pos()
# Direção da lanterna (vetor normalizado)
dir_x, dir_y = mouse_x - lanterna.lanterna_pos_x, mouse_y - lanterna.lanterna_pos_y
angle_lanterna = math.atan2(dir_y, dir_x) # Ângulo da lanterna

if lanterna.ligada:
    enemy_obj = enemy.get(Enemy)
    dx = enemy_obj.x - player.x
    dy = enemy_obj.y - player.y
    dist_sq = dx**2 + dy**2 # Distância ao quadrado

    if dist_sq < lanterna.raio ** 2: # Verifica se o inimigo está dentro do raio da lanterna
        angle_enemy = math.atan2(dy, dx) # Ângulo entre jogador → inimigo
        diferenca_angulo = abs(angle_enemy - angle_lanterna) # Diferença angular

        abertura = math.radians(30)

        if diferenca_angulo < abertura:
            enemy_obj.draw(screen) # Desenha o inimigo apenas se estiver no cone da lanterna
```

Figura 8: Estrutura de desenhar o fantasma somente quando a lanterna apontar para ele.